19
Automatic Deployment Using GitHub Actions and Azure

In the previous chapter, we wrote integration tests in our apps to ensure that the components can interact correctly. Now we will deploy ASP.NET Core and Vue.js apps to Azure Cloud services using the new **Continuous Integration** (**CI**)/**Continuous Deployment** (**CD**) tools, GitHub Actions. CI/CD automates the building, testing, and deployment of applications that saves time and improves the efficiency of application development.

In this chapter, we will cover the following topics:

- Introducing GitHub Actions – a CI/CD tool
- Understanding GitHub Actions
- Understanding where to deploy
- Automated deployment to Azure App Service using GitHub Actions

Technical requirements

Here is the link to the finished repository of this chapter: `https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter19/`.

Here is what you need to complete this chapter:

- **An Azure account**: `https://azure.microsoft.com/en-us/free/`
- **A GitHub account**: `https://github.com/`

Introducing GitHub Actions – a CI/CD tool

Do you still remember the methods of shipping apps into production before the arrival of CI/CD? There was copy and paste, FTP, scripting, or SSH when we deployed our apps to production. Then CI/CD came along, which helps to quickly build and publish applications without worrying about the manual configuration and deployment process. Then, a CI/CD tool called Hudson was born, which eventually became Jenkins. Finally, we started having tools such as Octopus Deploy, Visual Studio Team Services, Azure DevOps for deployment, and having better integration with our repositories. The CI/CD tools mentioned give us a single place where we can have our tools, all of our code, and the configuration needed for shipping our applications.

And then GitHub Actions came along.

Understanding GitHub Actions

GitHub Actions is a platform that automates software development workflows. The CI/CD pipeline is one of several workflows that you can use to automate with GitHub Actions.

GitHub Actions is a YAML-based workflow linked to getting repositories. This workflow can be triggered by webhooks or schedules, or by manually clicking a button to start a workflow.

A runner, where workflows run, comes in two forms. You have an option to use GitHub hosted or to do self-hosting. The GitHub hosted workflow provides different operating systems such as Ubuntu, macOS, or Windows, and they have pre-installed software, including several versions of .NET Core SDK.

GitHub Actions also generates logs while running. You can see the results of your build through the logs that are available in the GitHub portal.

Also, there are custom workflows available that have been built and shared by the community. Like the Azure team, some organizations have contributed and developed actions and workflows with different scenarios for us to use as building blocks in our workflows.

GitHub Actions for .NET apps

So how do we get our .NET Core applications wired up with GitHub Actions? There's a GitHub action for .NET that is called **setup-dotnet**. What is setup-dotnet? It provides a .NET CLI environment for a runner to specify the version of .NET to use. We can now use GitHub Actions in the `setup-dotnet` action to select a target version in our workflow.

We can also configure our environment to use private package repositories or sources. For example, if we have libraries or packages hosted as NuGet packages, GitHub packages, or Azure DevOps artifacts, we can securely leverage them by using actions.

Understanding where to deploy

You will see workflow templates later, with which you don't have to start writing your workflow file from scratch. You can deploy to Azure, AWS, GCP, IBM Cloud, Alibaba Cloud, OpenShift, and more.

We will focus on Azure since we are using C# and .NET Core. Here are the most common services in Azure:

- Azure App Service
- Azure Functions
- Azure Static Web Apps
- Azure Kubernetes Service

Let's find out what these are and their use cases.

When to deploy to Azure App Service?

Consider using Azure App Service if you encounter the following:

- When you have a web app from any programming languages and frameworks, or ASP.NET Razor pages, ASP.NET MVC, or an ASP.NET Web API, or even server-side Blazor
- When you want to deploy a dedicated high-performance application or a scalable application

- When you want more control over an environment and configuration without getting a whole virtual machine or an entire operating system, because App Service has rich feature sets such as slots, VNet support, and Azure Active Directory, to name a few, that are built into it

When to deploy to Azure Functions?

Azure Functions is part of the Azure serverless architecture space. So, when should you consider deploying to Azure Functions?

- When you want to run small stateless tasks, you can run functions inside Azure using a serverless architecture. However, there are also stateful and durable functions, which let us run complex scenarios.
- When you want an event-driven architecture with triggers and binding; for instance, blob files being dropped in an Azure Storage container.

When to deploy to Azure Static Web Apps?

Azure Static Web Apps, or **Azure SWA**, is a new service in Azure that allows you to have a static site. So, when should you consider deploying to Azure Static Web Apps?

- When you don't need a server at all to host your frontend application
- When you are using Blazor WebAssembly apps

Let's now check out Azure Kubernetes Service, or AKS.

When to deploy to Azure Kubernetes Service?

Azure Kubernetes Service, or **AKS**, is a managed Kubernetes service from Azure. So, when should you consider deploying to Azure Kubernetes Service?

- When you don't have any expertise in container orchestration but you want to use containers for deployment
- When you want to scale and manage Docker containers or other container-based apps using Kubernetes

These are your options when deploying to Azure.

So, GitHub Actions is a great solution for quickly setting up your CI/CD and GitHub repositories for building and deploying .NET Core applications. GitHub provides workflows, runners inside the GitHub portal, and viewable logs to make our lives easier. We have a plethora of available custom workflows whenever we want to deploy our ASP.NET Core apps to Azure or other types of applications to other known cloud services.

Now, let's move on to the next section to deploy an ASP.NET Core Web API with the **Progressive Web App** (**PWA**) Vue.js TypeScript app.

Automated deployment to Azure App Service using GitHub Actions

In this final section, we will deploy an ASP.NET Core 5 and Vue.js application to try an automated deployment to Azure App Service using GitHub Actions.

The ASP.NET Core 5 application will be a Web API project, while the Vue.js application will be built using TypeScript to show that we can add a TypeScript compiler to project's `csproj` file. We are also going to convert our Vue.js application to a PWA. If you are not yet familiar with PWAs, a PWA is a web application with mobile app features. PWAs can be installed on desktop, or mobile devices, and can be run offline.

We will use the existing ASP.NET Core and Vue app to simplify deployment to Azure using GitHub Actions. The goal here is to try deployment and focus only on GitHub Actions and the app service, which is the main topic in this chapter.

OK, so let's start:

1. Create a folder named `Travel` and then go inside the `Travel` folder using your command line and run the following `dotnet` CLI command:

   ```
   dotnet new sln
   ```

2. Create a `webapi` project by running the following command:

   ```
   dotnet new webapi -name WebUI
   ```

   A project named `WebUI` will be created.

3. Add the project to the .NET solution:

   ```
   dotnet sln add WebUI/WebUI.csproj
   ```

   The WebUI project is now visible on your IDE.

4. Go to the `WebUI` project directory:

```
cd WebUI
```

   While inside the `WebUI` project, we will create our Vue.js app here.

5. Run the following `vue` CLI command:

```
vue create client-app
```

   The preceding command will initiate the `vue` CLI for scaffolding of the Vue.js project.

6. Choose the **Manually select features** option, and then add **TypeScript**, **PWA**, and **Router**.

7. Remove the linter, press *Enter*, and then choose version `3.x`. Then, hit the *Enter* key on your keyboard several times to choose the default settings for the remainder of the configurations.

8. We rename the `client-app` folder inside the `WebUI` project to `ClientApp`.

   If you are familiar with the ASP.NET Angular and ASP.NET React templates, you will notice the templates and the `ClientApp` naming convention in the naming of the SPA folder.

9. After renaming the folder of the Vue app, let's update `WebUI.csproj` of the project with the code from the following GitHub repository: `https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter19/Travel/WebUI/WebUI.csproj`.

   The code from the preceding link is similar to the `csproj` file of the `Travel.WebApi` project. The only difference is that we are adding three extra properties below the `TargetFramework` property like so:

```
    <TypeScriptCompileBlocked>true</
    TypeScriptCompileBlocked>
    <TypeScriptToolsVersion>Latest</
    TypeScriptToolsVersion>
    <IsPackable>false</IsPackable>
```

   `TypeScriptCompileBlocked` is set to `true` to help us debug our Vue TypeScript application.

   `TypeScriptToolsVersion` targets the latest TypeScript version.

   `IsPackable`, which is set to `false`, will not package the project.

10. Next, we update the `Startup.cs` file for the `WebUI` project with the code from the following link: `https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter19/Travel/WebUI/Startup.cs`.

    The code from the preceding link is also simple to help us quickly deploy our application. It is a simple Web API project importing `VueCliMiddleware` along with the `SpaStaticFiles` service.

11. Next, we run the following command inside the `WebUI` project:

```
dotnet run
```

    The preceding command will build and run the backend and frontend applications. You should see the `webpack` progress reach `100%`, as shown in the following screenshot:

```
[webpack.Progress] 100%
 App running at:
 - Local:   http://localhost:8080/
 It seems you are running Vue CLI inside a container.
 Access the dev server via http://localhost:<your container's
external mapped port>/
 Note that the development build is not optimized.
 To create a production build, run npm run build.
Issues checking in progress...
No issues found.
```

Figure 19.1 – Vue app build and running

Figure 19.1 shows that the application is running without any problems. You can also check `https://localhost:5001` to see that the application is running.

12. Publish the project to your own GitHub account and put it in a private repository. After publishing your `Travel` project to GitHub, go to the **Actions** tab menu on your project's repository page.

You should see the suggested workflows as shown in *Figure 19.2*, and the other workflows for deployment, continuous integration, and other steps. These are the workflows that will make our lives easier:

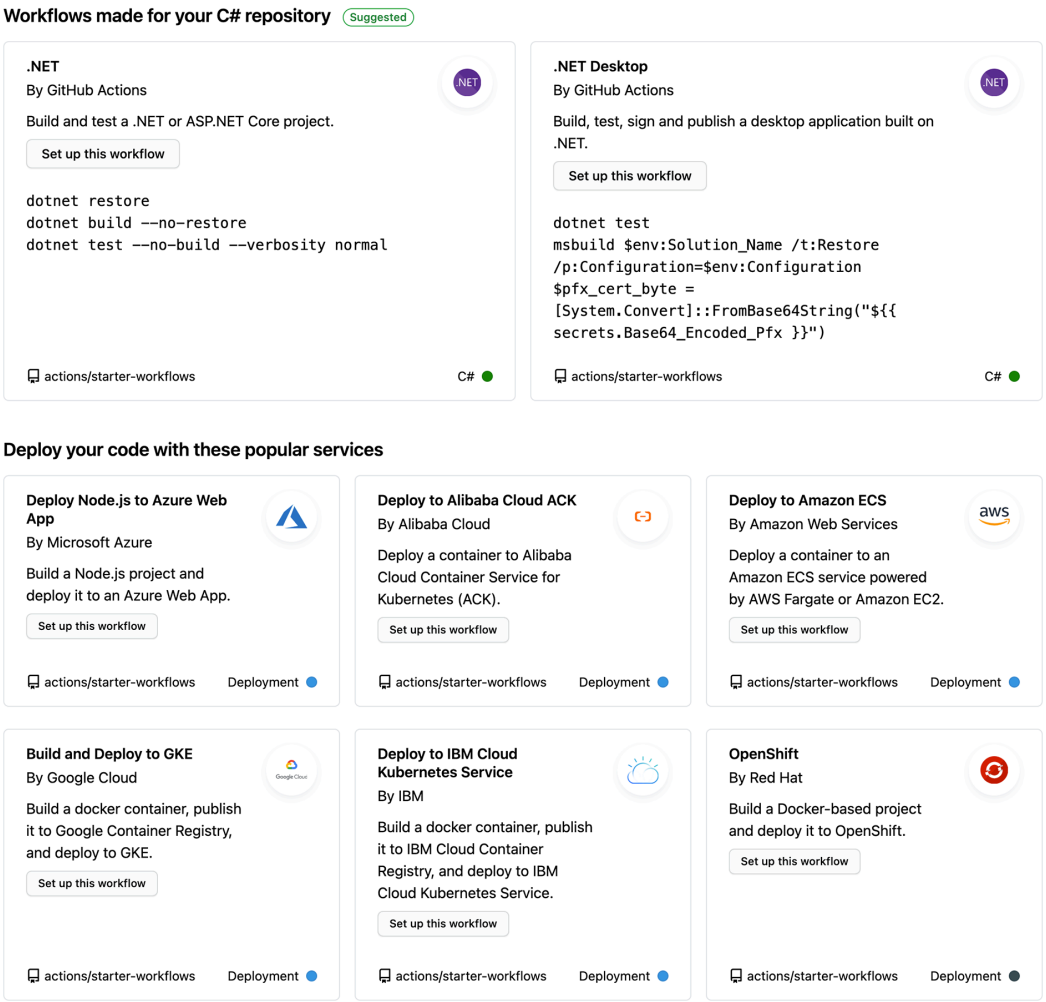

Figure 19.2 – Cloud services in GitHub Actions

13. Now, click on **Set up this workflow** under the **.NET** section. Don't click on **Commit** because we will not configure our workflow here; we will let Azure App Service do this for us:

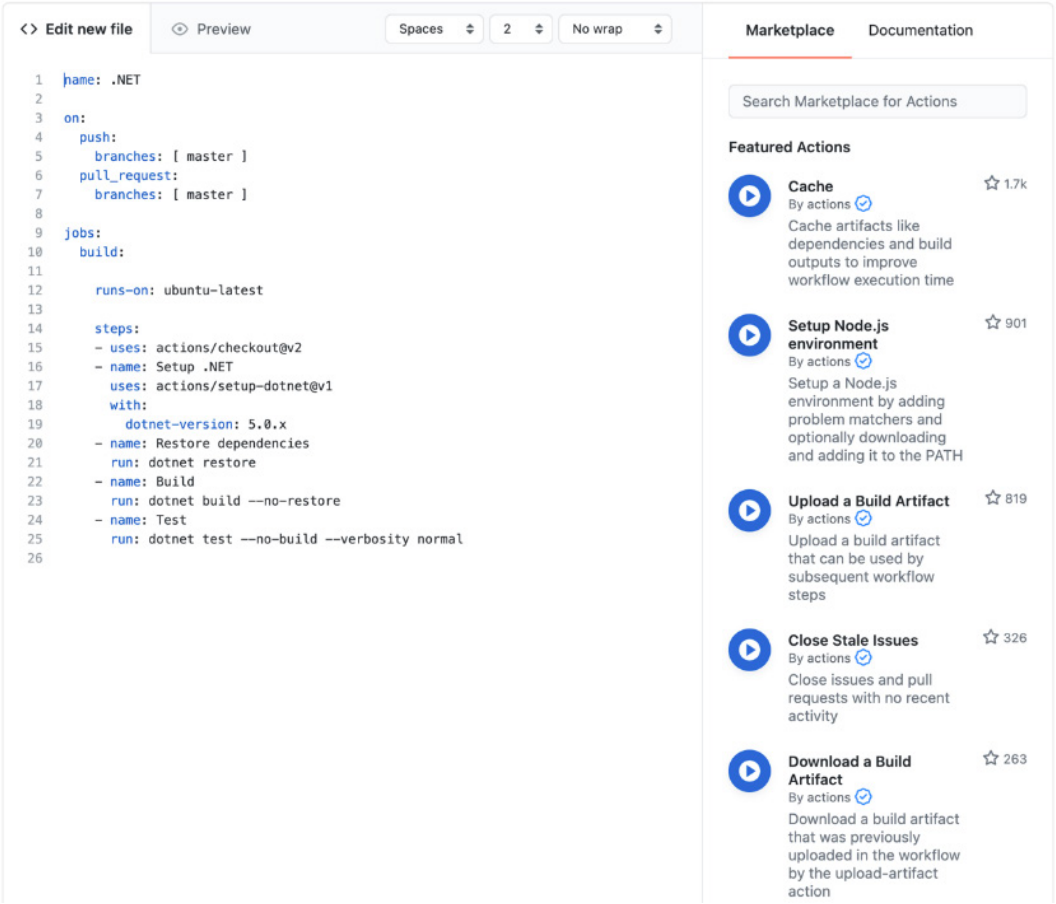

Figure 19.3 – Sample ready-made workflow

The preceding *Figure 19.3* is a pre-configured workflow that we can adjust.

Now let's check out the workflow file.

Syntax of the workflow file

Let's go and learn the basic syntax of the workflow file to understand how to write our configuration of the workflow.

The `optional name` attribute is where you describe what the workflow is doing.

`on` is where we declare events. There are `push` and `pull_request` events already in place here. `push` triggers the jobs every time someone pushes to the master branch, and `pull_request` triggers the jobs again every time a `pull` request gets created with the master branch. An excellent example is to test our application to ensure that the pull request is mergeable. You can use more events by going to `https://docs.github.com/en/actions/reference/events-that-trigger-workflows` to see the detailed explanations and usages of different events.

Now let's move on to jobs. A job contains a sequence of tasks called **steps**.

Jobs

Jobs are groups of actions. There can be one or more jobs you can define in a workflow file; for instance, one for `build`, one for `test`, and another one for `deploy`. The jobs could be arbitrary, like the name of the workflow. The example runs on `ubuntu-latest`, which is the agent that runs on GitHub. The first step or task is `actions/checkout@v2` and it checks out the code before running a build or a test.

The actions path in `actions/checkout@v2` means that it is one of the predefined actions hosted in GitHub. This is amazing because we don't have to create the action ourselves. But wait, there's more. You can go to `https://github.com/actions` to see the list of different repositories containing all the actions you might need. You can also check out the documentation of actions (`https://docs.github.com/en/actions` with some code examples).

The second step uses the action called `setup-dotnet`, which is another repository in this actions list. This action essentially prepares your environment with a specific version of `dotnet` defined in the file. Unlike in Jenkins, you don't have to install or configure anything in GitHub Actions by specifying what you want in your environment.

As you notice, you use actions by defining the actions in the uses. To run commands, you can use the `run` attribute. In the sample workflow, the step for `build`, also known as the **build step**, triggers the `dotnet build --no-restore` command, and the step for `test`, also known as the **test step**, runs the `dotnet test --no-build --verbosity normal` command.

All the steps are done in the same environment in the flow where your code gets checked out, the dotnet version gets installed, and then you call `dotnet build` and `dotnet test` in the same environment.

Moving on, let's now create an Azure App Service in Azure Portal.

Creating an Azure App Service instance in the Azure portal

We are going to create an instance of Azure App Service so that we can easily deploy our app to Azure:

1. Go to Azure App Service in `https://portal.azure.com/#home`:

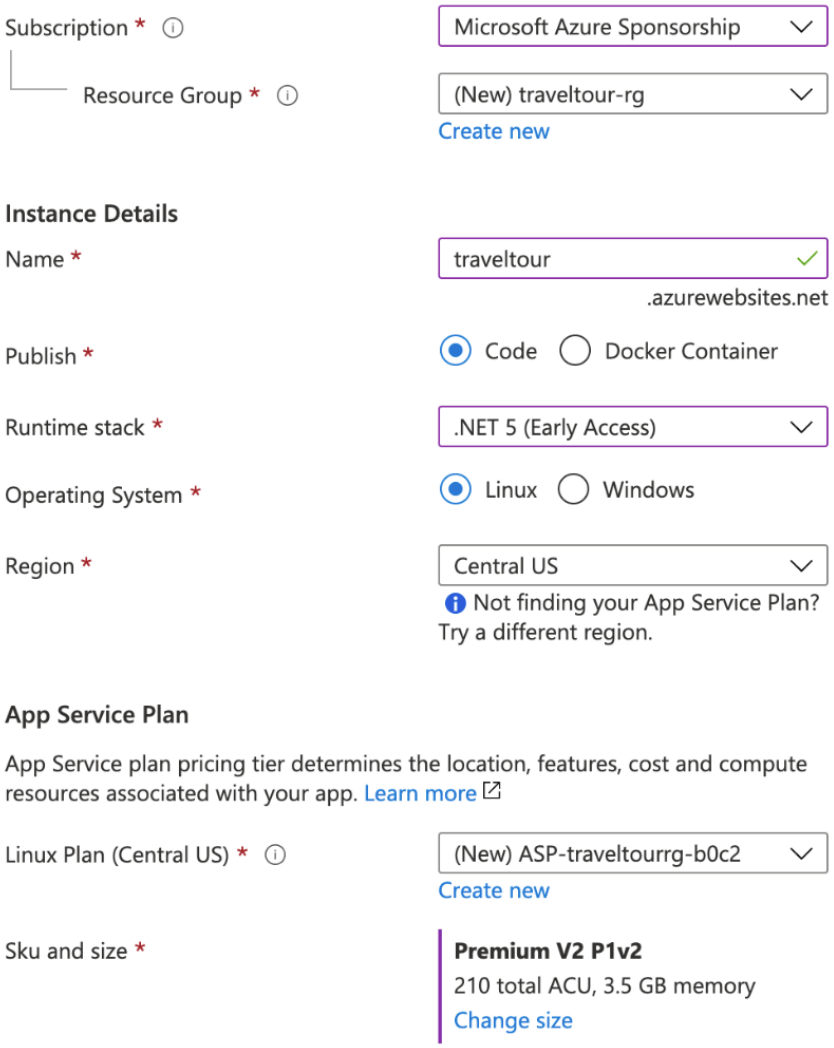

Figure 19.4 – Creating an App Service instance in Azure

The preceding *Figure 19.4* shows the details of the App Service instance we are going to create. We will publish the app using code and the .NET 5 runtime. Hit the **Create** button that you will see after adding the configuration.

2. Now go to the Azure App Service instance that has been created and click on **Deployment Center**:

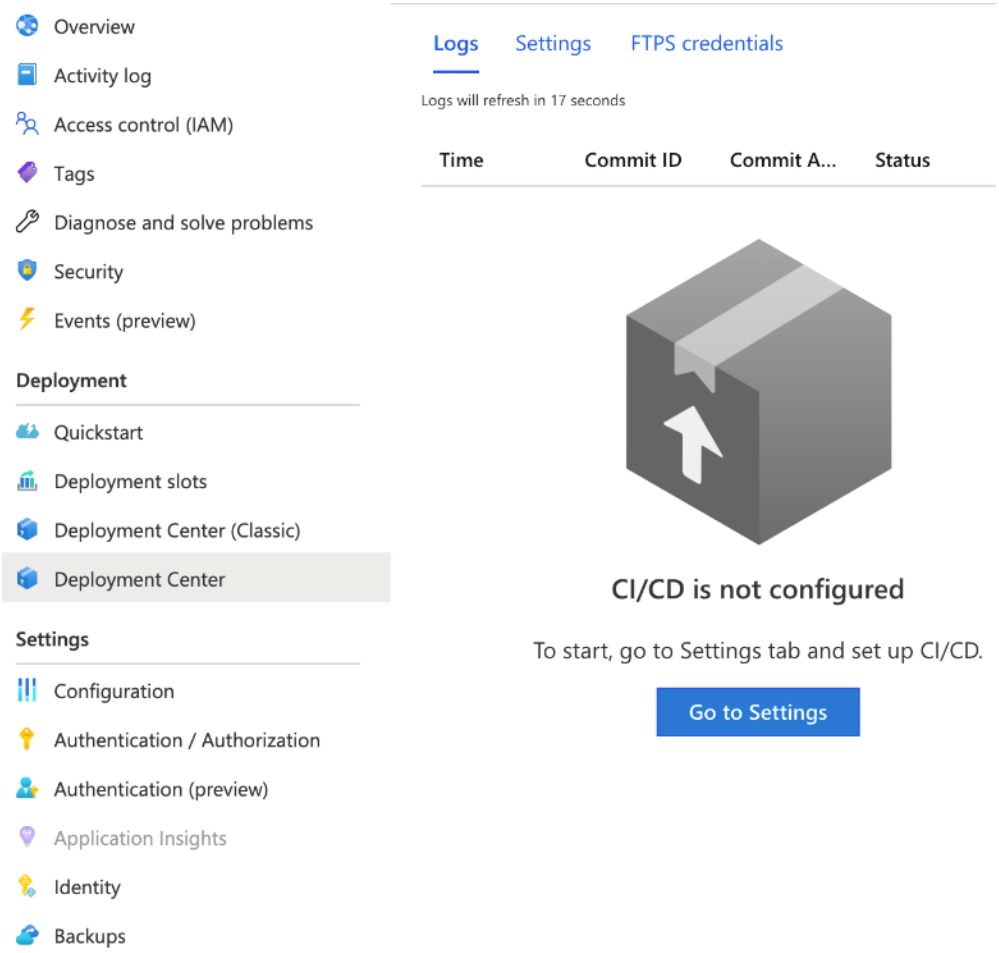

Figure 19.5 – The deployment center of App Service

Figure 19.5 shows that **CI/CD is not configured**. Let's configure the CI/CD by going to the settings.

3. Add the private GitHub repository you created earlier:

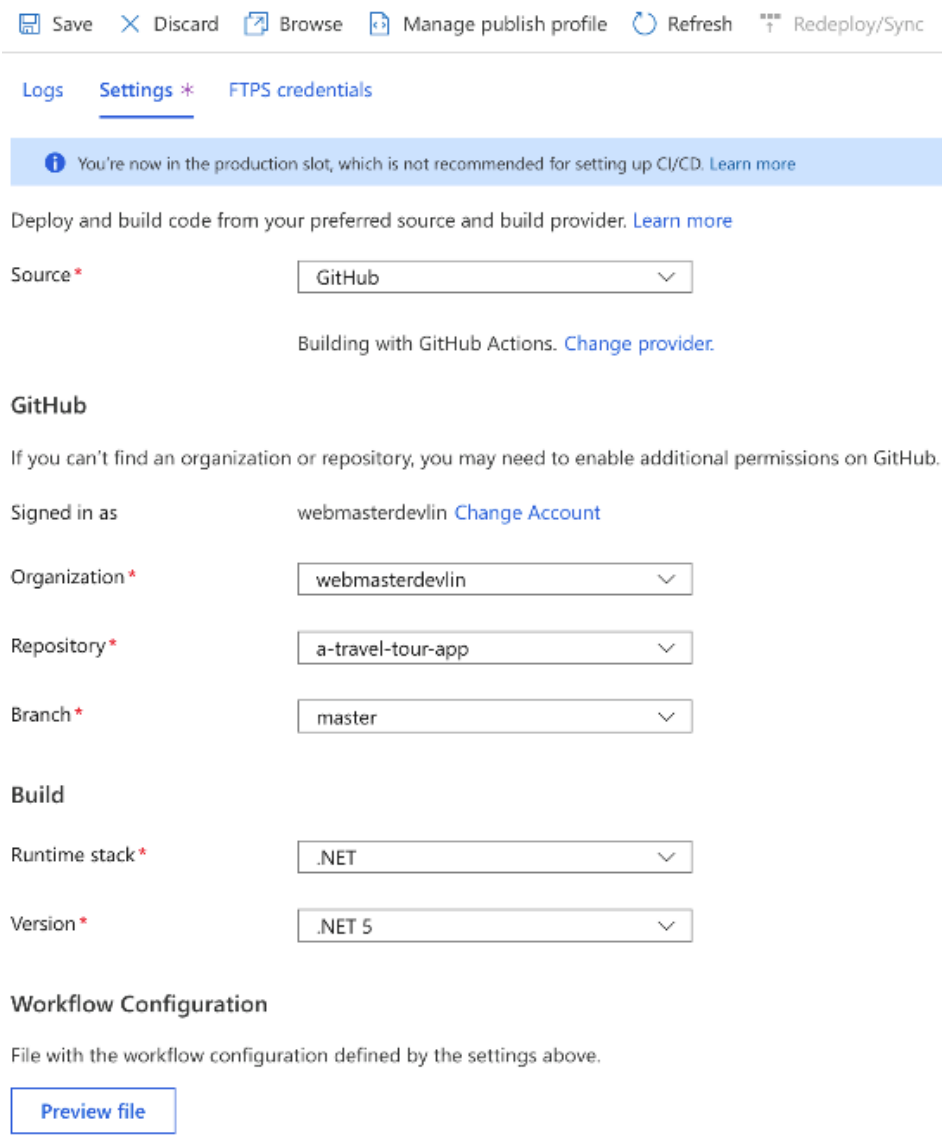

Figure 19.6 – Adding the private GitHub repository

Figure 19.6 shows how we can add the private .NET project we created earlier. We are also using .NET 5 as our runtime.

4. Then, click **Preview file** to see the workflow:

Figure 19.7 – Workflow YAML configuration file

Figure 19.7 shows five steps in the workflow YAML configuration file that App Service created for us. Since the YAML configuration file is part of the code or repository, we should not put plain text credentials in the file. We can use placeholders that will reference secrets. I'll show you later where the secrets are located in GitHub.

5. Now click **Save** and go to the project's GitHub repository.
6. Let's check out the **Actions** tab to see whether App Service could add the workflow file:

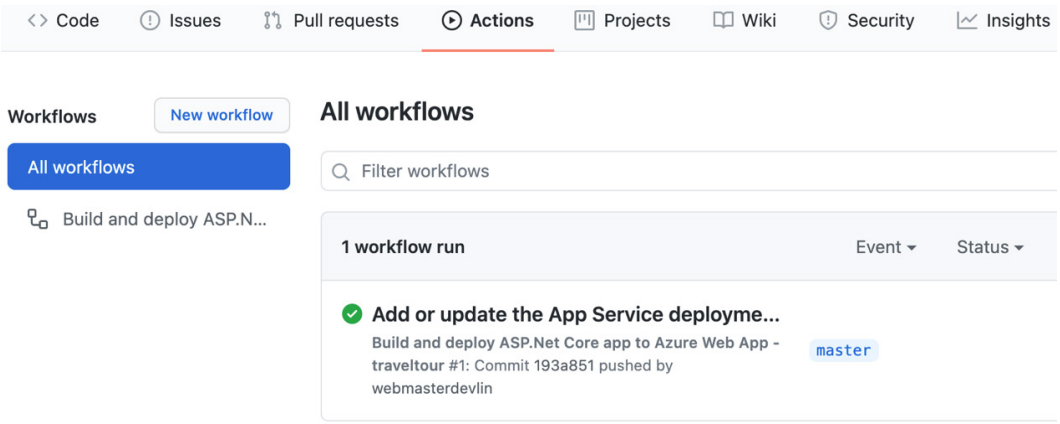

Figure 19.8 – Workflow run

Figure 19.8 shows a workflow with a green icon, which means that the application has been built successfully and deployed after waiting for a few seconds.

7. Let's click the workflow to see the jobs summary:

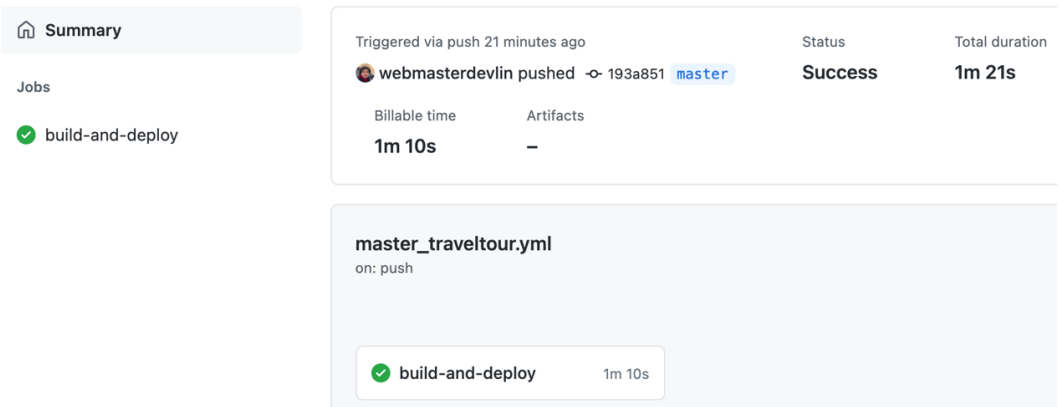

Figure 19.9 – Summary of jobs

Figure 19.9 shows the status and total duration of the workflow.

8. Now let's click on **build-and-deploy** to see the steps associated with the workflow job:

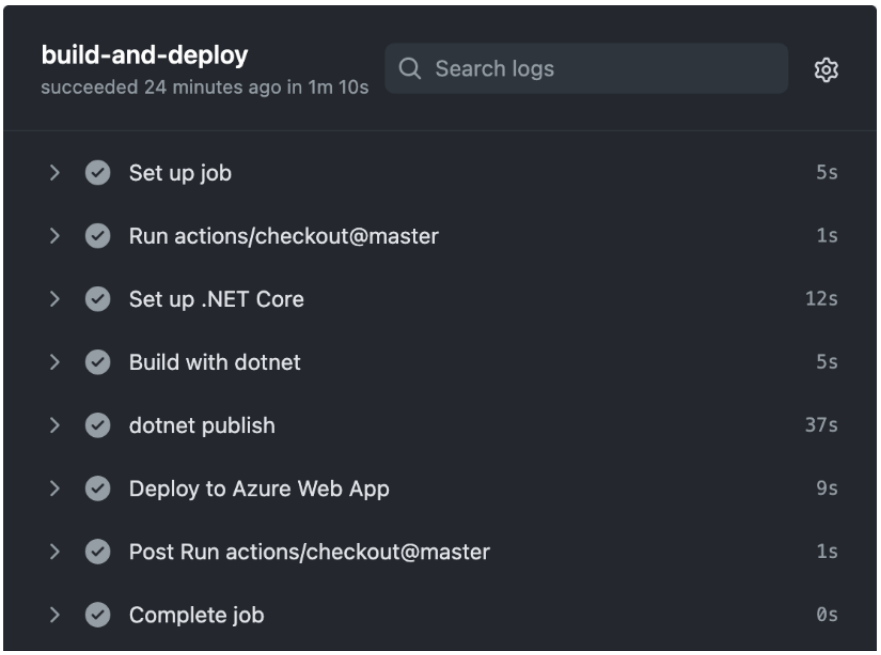

Figure 19.10 – A build-and-deploy workflow job

Figure 19.10 shows the five steps of the workflow job. The steps are as follows:

Run actions/checkout@master

Set up .NET Core

Build with dotnet

dotnet publish

Deploy to Azure Web App

You will also notice that there are also steps before and after running the configuration file's five steps.

9. Now go to App Service and click the **Overview** menu:

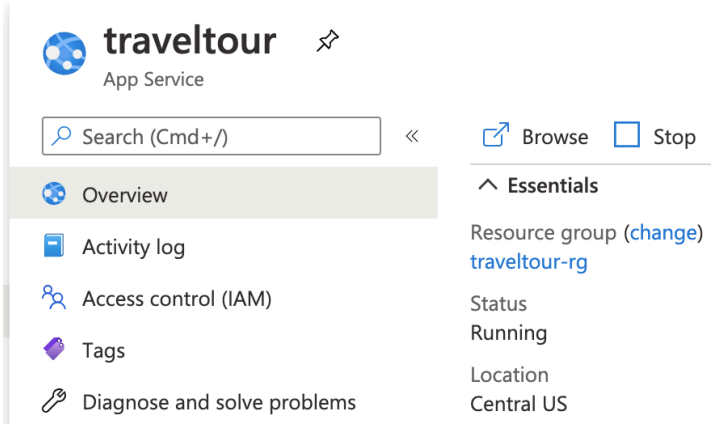

Figure 19.11 – App Service overview

Figure 19.11 shows an overview of App Service.

10. Now click the **Browse** link that will redirect us to the application's URL. You should see that Vue.js is running in the browser:

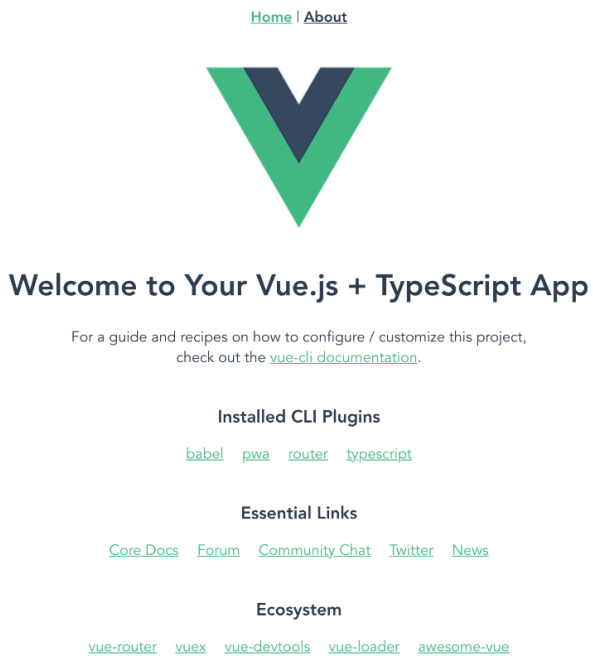

Figure 19.12 – Vue.js + TypeScript App

Figure 19.12 shows **Vue.js + TypeScript App**, which we created earlier.

11. Let's check out the end of the URL bar to see whether we can install the application:

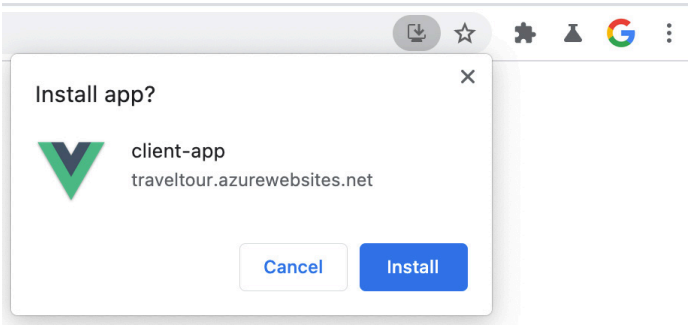

Figure 19.13 – The installable PWA Vue.js app

Figure 19.13 shows that the Vue.js app is installable, meaning it is a PWA app and will still work in offline mode.

12. Now, go to the **Settings** tab of the GitHub repository and click the **Secrets** menu:

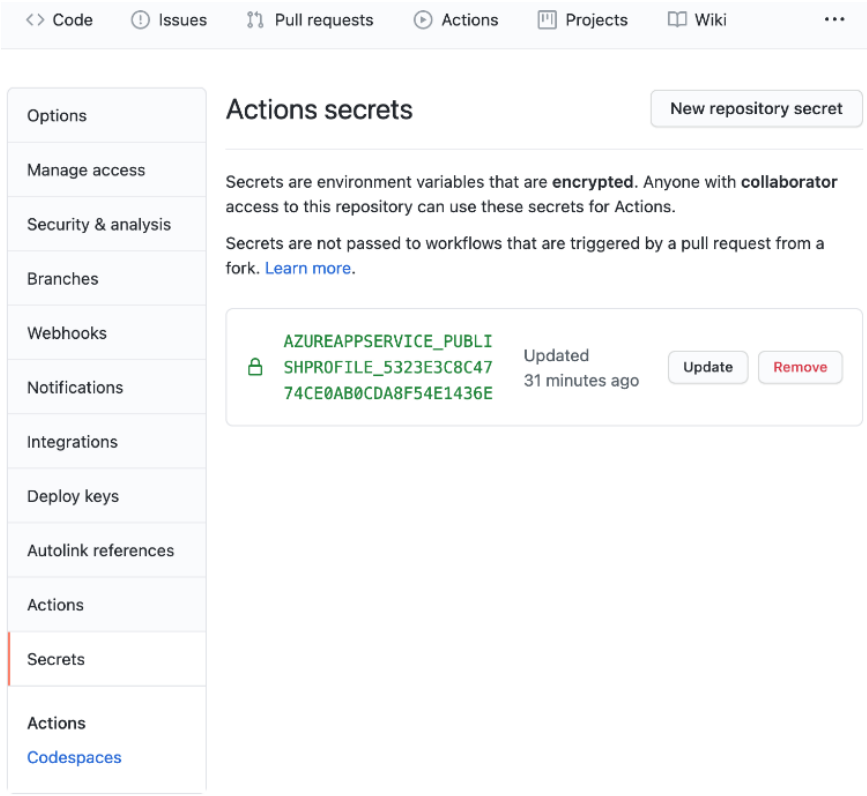

Figure 19.14 – Actions secrets

Figure 19.14 shows where we can safely keep the sensitive keys and secrets of our applications.

Now let's test the CI/CD capability of GitHub Actions and the workflow that we have set up.

Testing the CI/CD capability of GitHub Actions and the workflow

Here, we will see whether our CI/CD configuration and set up in GitHub Actions will satisfy our needs:

1. Run the `git pull` command to bring the changes in your GitHub repository down to your local machine.
2. Update the `Welcome to Your Vue.js + TypeScript App` text or message like so:

Figure 19.15 – Edited HelloWorld msg props

Figure 19.15 shows the edited `msg` props of the `HelloWorld` component in the `Home.vue` file.

3. Next, we commit and push your changes to the master. Go to GitHub Actions to see that the workflow was triggered. Wait for the build and deploy job to finish:

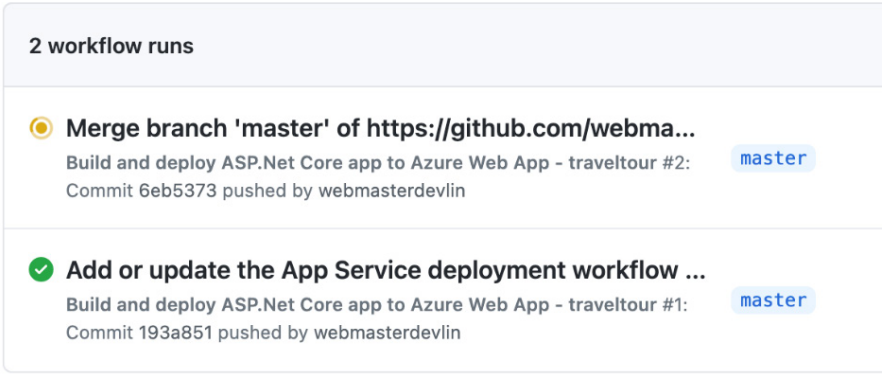

Figure 19.16 – Testing CI/CD

Figure 19.16 shows that the CI functionality is working, followed by the CD functionality after a few seconds.

4. Open a new tab in the browser and go to the web app's URL to see that the changes we made in the application are now reflected on the internet:

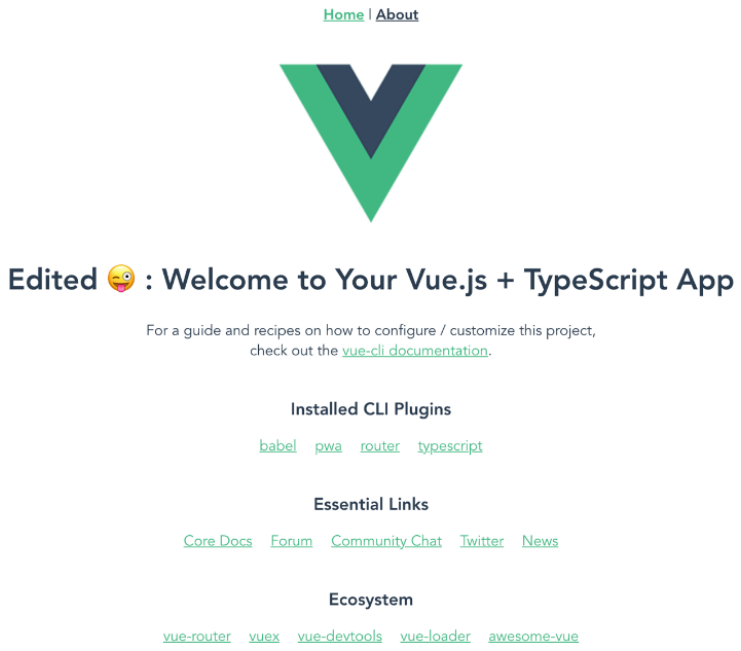

Figure 19.17 – Showing the updated Vue.js + TypeScript app

Figure 19.17 shows the **Edited** word and the emoji we included in the changes.

5. Now, go to `{yourSubDomain}.azurewebsites.net/weatherforecast` to see whether the Web API's `WeatherForecast` endpoint can respond with JSON data:

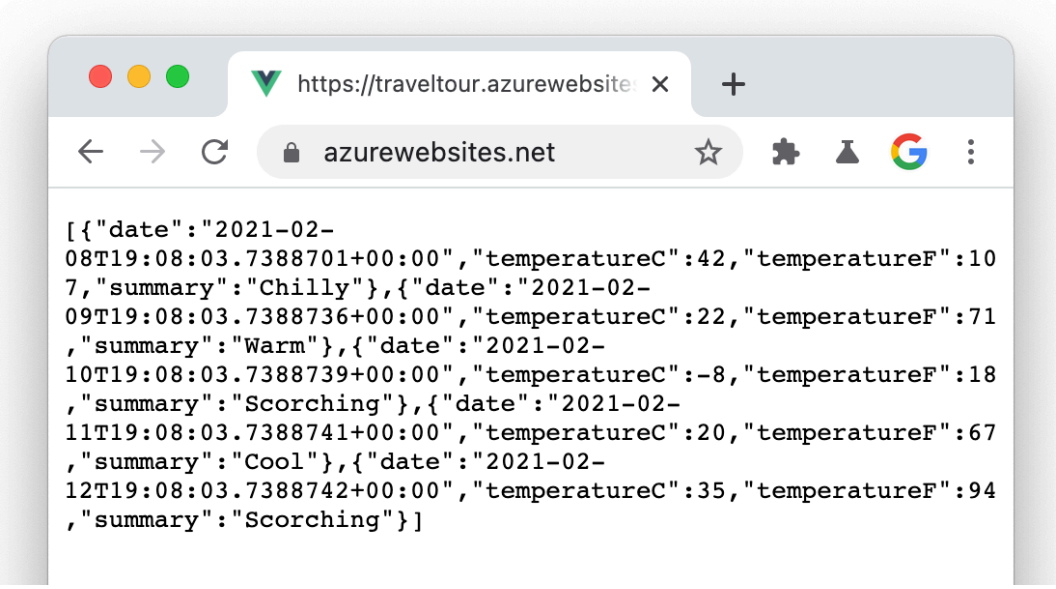

Figure 19.18 – Response from the WeatherForecast endpoint

Figure 19.18 shows that the `WeatherForecast` endpoint responds with an array in JSON format.

The Vue.js TypeScript and the controller of the ASP.NET Core Web API project are working correctly in CI/CD using GitHub Actions.

Now let's summarize what you have learned.

Summary

That's a wrap. Here are some takeaways from this chapter. You learned what GitHub Actions is and how easy deployment can be using GitHub Actions. You learned the right time and use cases to deploy to Azure App Service, Azure Functions, Azure SWA, and Azure AKS. You also learned how to deploy to Azure App Service using GitHub Actions, including the workflow's YAML configuration, the available actions, and CI/CD.

So you've made it this far. Thank you for finishing the book, and I am proud of you and your enthusiasm for learning new tools and things. You can apply what you have learned here in a project, given the requirements of your project match the problems and solutions you have learned from the book.

The course has taught you how to architect an ASP.NET Core application and a Vue.js application like a senior developer, bringing value to your customers or clients.

The next step that I will suggest for you is to get a new Packt book about a standalone ASP.NET Core or Vue.js topic to solidify what you have learned from this book.

On behalf of the whole Packt team and editors, we wish you all the best in all stages of your career and life.